TUGAS KULIAH KELOMPOK
Assignment Group Case

Persiapan Lingkungan Pengembangan

Development Environment Preparation

Proyek SisaKulkas: deteksi bahan makanan dengan YOLOv8 dan rekomendasi resep masakan Indonesia

Mata Kuliah	Proyek Perangkat Lunak (Software Project)
Sesi Pertemuan	6
Judul Materi	Pengujian perangkat lunak (Software testing)
Kategori	Tugas Proyek (Project Case)
Nama Dosen	Anggun Fergina, M.Kom
Tugas	Menentukan bahasa pemrograman dan framework, setup repository GitHub/GitLab, menyiapkan server dan database awal
Tautan repository	..... (diisi tim setelah repository dibuat)

Anggota Kelompok

No	Nama	Peran dalam proyek
1	Pirni	Project Manager (ketua)
2	Nazwa Azzahra	Data Engineer (dataset, database, server)
3	M. Denindra Pratama	ML Engineer (training dan ekspor model)
4	Shelva Meriana N	Developer aplikasi Flutter
5	M. Rizky Akbar	QA dan Dokumentasi

Dikumpulkan dalam bentuk PDF dan dipresentasikan pada Sesi 6. Oktober 2026.

1. Ringkasan

SisaKulkas adalah aplikasi mobile yang mengenali bahan makanan dari foto (kamera atau galeri), lalu menampilkan masakan Indonesia yang bisa dibuat dari bahan tersebut. Dokumen ini melaporkan hasil persiapan lingkungan pengembangan: pilihan teknologi, struktur repository, serta server dan database awal yang **sudah dijalankan dan diuji**. Bagian yang masih berupa rencana atau belum diuji ditandai dengan jelas.

Butir tugas	Hasil	Status
Bahasa pemrograman dan framework	Dart + Flutter (aplikasi), Python + Flask (server), SQL (database), Python + Ultralytics (model)	Ditetapkan
Repository GitHub	Repository tunggal (monorepo) dengan 4 folder, aturan branch, templat PR dan issue, 2 workflow CI	Struktur siap. Push ke GitHub dan tautan diisi tim
Server awal	Flask + Gunicorn dengan 7 endpoint	Berjalan dan diuji (12 tes lolos)
Database awal	SQLite (pengembangan) dan PostgreSQL (produksi), 5 tabel, data awal 24 bahan dan 26 resep	Berjalan dan terisi. PostgreSQL belum diuji
Aplikasi mobile	Flutter, model ONNX berjalan di perangkat	Logika inti diuji. Build dan uji di HP belum dilakukan

2. Bahasa Pemrograman dan Framework

2.1 Pilihan tim

Lapisan	Teknologi	Fungsi	Alasan
Aplikasi pengguna	Dart, Flutter	Tampilan, kamera dan galeri, kotak deteksi, daftar resep	Satu kode untuk Android dan iOS. Tampilan konsisten dan performa baik
Inferensi di perangkat	ONNX Runtime (paket onnxruntime_plus)	Menjalankan model YOLOv8 di HP	Memakai model best.onnx yang sudah ada. Berjalan offline dan foto tidak dikirim ke server
Model deteksi	Python, Ultralytics YOLOv8, Roboflow, Google Colab	Menyiapkan dataset, melatih dan mengekspor model	Ekosistem machine learning paling lengkap dan sudah dikuasai tim
Server	Python, Flask, Gunicorn	API data resep dan riwayat deteksi	Ringan dan satu bahasa dengan bagian model
Akses database	SQLAlchemy (ORM)	Kode yang sama untuk SQLite dan PostgreSQL	Pindah database tanpa mengubah kode
Pengujian	pytest, flutter test, skrip uji Dart	Uji otomatis server dan logika inti	Mudah dijalankan dan dipakai di CI
Versi kontrol dan CI	Git, GitHub, GitHub Actions	Kerja tim, tinjauan kode, tes otomatis	Gratis untuk repository mahasiswa

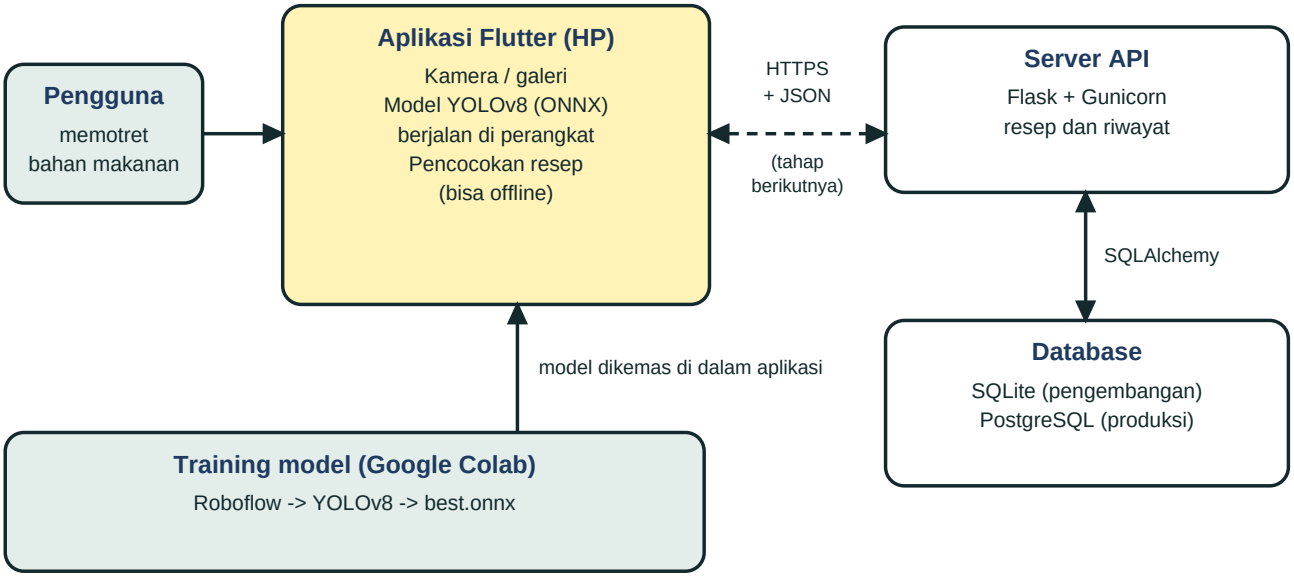
Versi yang dipakai saat pengujian: Python 3.12, Flask 3.1, Flask-SQLAlchemy 3.1, Gunicorn, pytest 9, Dart SDK 3.13. Paket Flutter: onnxruntime_plus ^1.5.0, image_picker ^1.1.2, image ^4.2.0.

2.2 Perbandingan dengan contoh pada tugas

Contoh	Jenis	Alasan tidak dipilih untuk proyek ini
Laravel, CodeIgniter	Framework web PHP	Kebutuhan utama adalah aplikasi mobile dengan deteksi di kamera. Model dilatih dengan Python sehingga PHP menambah bahasa kedua tanpa manfaat jelas
Node.js	Runtime JavaScript untuk server	Bisa dipakai, tetapi tim sudah memakai Python untuk model. Satu bahasa di server dan model lebih mudah dirawat
React	Library antarmuka web	Menghasilkan web, sedangkan tugas proyek berupa aplikasi mobile. Flutter menghasilkan aplikasi Android dan iOS dari satu kode

Sebelum memilih Flutter, tim membuat prototipe web (JavaScript) untuk membuktikan bahwa model dan logika resep bekerja. Logika yang sama kemudian diterapkan ulang di Flutter dan server, dan hasilnya dibandingkan agar identik (lihat bagian 7).

3. Arsitektur Sistem



Deteksi bahan berjalan di dalam aplikasi, sehingga foto tidak meninggalkan perangkat dan aplikasi tetap bisa dipakai tanpa internet. Server dan database menjadi sumber data resep serta penyimpan riwayat deteksi, sehingga resep bisa ditambah tanpa memperbarui aplikasi. Sambungan aplikasi dengan server (sinkronisasi resep dan pengiriman riwayat) **belum dibuat** dan menjadi langkah berikutnya, karena itu garisnya putus-putus pada gambar.

4. Setup Repository

Platform: GitHub, repository privat bernama **sisakulkas** (monorepo). Seluruh anggota diundang sebagai collaborator, Project Manager sebagai admin.

4.1 Struktur folder

```
sisakulkas/
  app_flutter/          aplikasi mobile (Dart, Flutter) + model best.onnx
  server/               API dan database (Python, Flask, SQLAlchemy)
  ml/                   notebook training dan ekspor model (Colab)
  docs/                 laporan dan dokumen tugas
  .github/
    workflows/          ci-server.yml (tes otomatis), build-apk.yml (membangun APK)
    pull_request_template.md, ISSUE_TEMPLATE/
  README.md  CONTRIBUTING.md  .gitignore
```

4.2 Aturan kerja tim

- **main** hanya berisi kode stabil dan tidak boleh di-push langsung. **develop** adalah cabang integrasi. Fitur dikerjakan di **feature/nama-fitur**.
- Setiap perubahan lewat pull request dan ditinjau minimal satu anggota lain. Tes CI harus lolos sebelum digabung.
- Pesan commit memakai awalan **feat, fix, docs, test, chore**, misalnya "feat: tambah endpoint rekomendasi".
- Tugas dicatat di GitHub Issues dengan papan Projects. Templat issue (tugas, bug) dan templat pull request sudah disediakan.
- Rahasia (kunci API, kata sandi, file .env) tidak boleh masuk repository. Contoh konfigurasi disimpan di .env.example.
- File model (best.onnx, 12 MB) berada di bawah batas 100 MB per file GitHub sehingga dapat di-commit biasa. Jika model membesar, gunakan Git LFS.

4.3 Langkah setup (dikerjakan Project Manager)

```
cd sisakulkas
git init -b main
git add .
git commit -m "chore: struktur awal proyek"
git remote add origin https://github.com/<akun-github>/sisakulkas.git
git push -u origin main
git switch -c develop
git push -u origin develop
```

Setelah push: (1) undang anggota di Settings > Collaborators, (2) aktifkan Branch protection pada **main** (wajib pull request dan satu persetujuan), (3) buat papan Projects dan label per peran.

4.4 Integrasi berkelanjutan (CI)

Dua workflow GitHub Actions sudah ditulis dan sintaksnya divalidasi. **ci-server.yml** menjalankan pytest setiap ada perubahan di folder server. **build-apk.yml** membangun APK dari aplikasi Flutter secara manual (tombol Run workflow). Keduanya **belum dijalankan di GitHub** karena repository belum di-push.

5. Server Awal

Server berupa API REST berbasis Flask yang dijalankan dengan Gunicorn. Konfigurasi dibaca dari environment variable (**DATABASE_URL**), ukuran permintaan dibatasi 1 MB, dan seluruh masukan divalidasi. Server tidak menyimpan foto, hanya daftar bahan.

5.1 Struktur dan endpoint

```
server/
  app/__init__.py     pabrik aplikasi, perintah "flask seed"
  app/config.py       konfigurasi (DATABASE_URL, batas ukuran)
  app/models.py       model SQLAlchemy (5 tabel)
  app/routes.py       endpoint API
  app/matcher.py      logika pencocokan bahan dengan resep
  app/seed.py         pengisi data awal
  data/               recipes.json, config.json (sumber data awal)
  tests/              12 tes pytest
  wsgi.py  requirements.txt  .env.example  README.md
```

Metode	Alamat	Fungsi
GET	/api/health	Cek server hidup, jumlah resep dan bahan
GET	/api/ingredients	Daftar bahan yang dikenali model
GET	/api/recipes	Daftar resep
GET	/api/recipes/{id}	Detail resep beserta langkah memasaknya
POST	/api/recommend	Menerima daftar bahan, mengembalikan resep terurut dari yang paling cocok
POST	/api/history	Menyimpan riwayat hasil deteksi
GET	/api/history?limit=20	Riwayat terbaru

5.2 Cara menjalankan

```
cd server
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
flask --app wsgi seed                    # isi data awal (aman diulang)
gunicorn -b 0.0.0.0:5000 -w 2 wsgi:app   # jalankan server
pytest                                   # jalankan tes
```

5.3 Bukti: server berjalan dengan Gunicorn

Berikut keluaran nyata dari server yang dijalankan di lingkungan pengembangan (Gunicorn, SQLite).

```
$ flask --app wsgi seed
Seed selesai: {'bahan': 24, 'resep': 26, 'resep_baru': 26, 'langkah': 99}
$ flask --app wsgi seed        # dijalankan ulang: tidak ada duplikasi
Seed selesai: {'bahan': 24, 'resep': 26, 'resep_baru': 0, 'langkah': 99}

GET /api/health
{"status":"ok","resep":26,"bahan":24}

POST /api/recommend
{"ingredients": ["Telur", "Bawang Merah", "Cabe"], "min_score": 0.6}
-> 200 OK, 4 resep; dua teratas:
{"id": 2, "nama": "Telur Dadar", "skor": 1.0, "siap": true, "ada": ["Telur", "Bawang Merah"],
  "kurang": []}
{"id": 21, "nama": "Nasi Goreng Telur", "skor": 0.75, "siap": false, "ada": ["Telur", "Bawang
  Merah", "Cabe"], "kurang": ["Bawang Putih"]}

POST /api/recommend   {"ingredients": ["Naga"]}
-> 422  {"error":"Bahan tidak dikenali: ['Naga']"}

POST /api/history     {"bahan": ["Telur", "Cabe"], "jumlah_objek": 3}
-> 201  {"id":1}

GET /api/history
{"id": 1, "waktu": "2026-10-26T03:58:06.744803+00:00", "bahan": ["Telur", "Cabe"], "jumlah_objek":
  3}
```

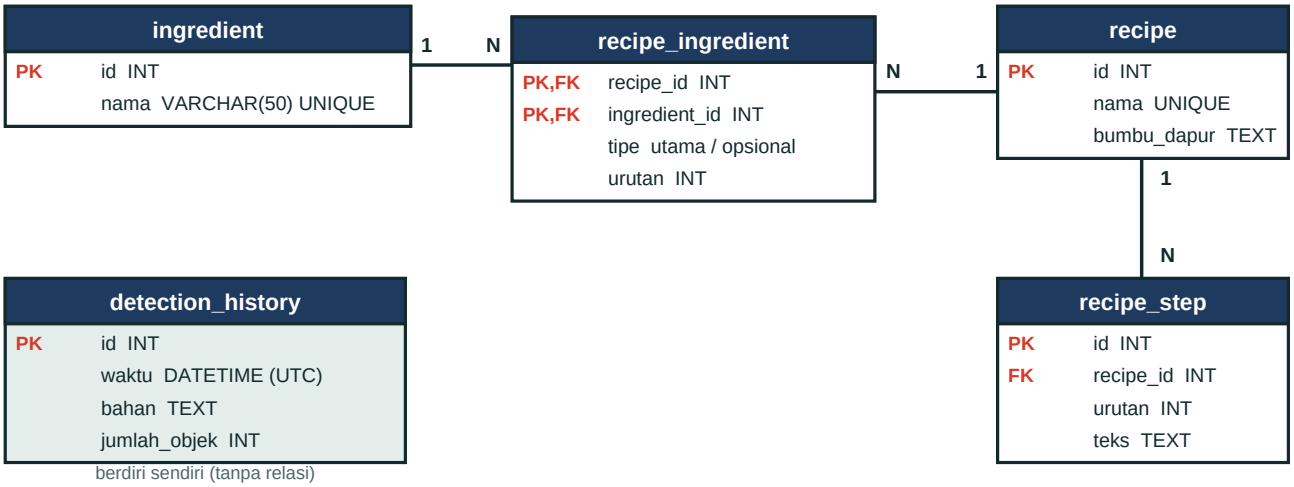
5.4 Keamanan dan hosting

- Validasi masukan di semua endpoint: tipe data, daftar bahan harus dikenal (tidak peka huruf besar-kecil), rentang angka dibatasi, JSON tidak valid ditolak.
- Kata sandi dan alamat database dibaca dari environment variable. Belum ada autentikasi pengguna karena tidak dibutuhkan pada tahap ini.
- Server saat ini berjalan di lingkungan lokal. Pilihan hosting untuk produksi belum diputuskan. Catatan: layanan Hugging Face Spaces bertipe Docker kini berbayar, sehingga tim perlu membandingkan opsi lain (VPS kecil atau layanan PaaS) beserta ketentuan gratisnya. HTTPS wajib saat deployment.

6. Database Awal

DBMS: SQLite untuk pengembangan (satu file, tanpa instalasi) dan PostgreSQL untuk produksi. Keduanya diakses lewat SQLAlchemy sehingga kode tidak berubah; cukup mengganti **DATABASE_URL**. Pengujian dilakukan pada SQLite. **Koneksi PostgreSQL belum diuji** pada tahap ini.

6.1 Diagram relasi (ERD)



6.2 Rincian tabel

Tabel	Kolom	Keterangan
ingredient	id, nama	Bahan makanan. Nama sama persis dengan nama kelas model YOLOv8, sehingga hasil deteksi langsung cocok dengan resep
recipe	id, nama, bumbu_dapur	Data masakan. Nama resep unik
recipe_ingredient	recipe_id, ingredient_id, tipe, urutan	Bahan tiap resep. Tipe berisi "utama" atau "opsional" (dibatasi CHECK). Kolom urutan menjaga urutan penulisan bahan
recipe_step	id, recipe_id, urutan, teks	Langkah memasak berurutan. Dihapus otomatis jika resepnya dihapus
detection_history	id, waktu, bahan, jumlah_objek	Riwayat hasil deteksi (waktu dalam UTC). Foto tidak disimpan

6.3 Data awal

Data awal diisi dari **recipes.json** dan **config.json** lewat perintah **flask seed**. Perintah ini idempoten: dijalankan dua kali tidak menggandakan data (terbukti pada bagian 5.3, seed kedua menambah 0 resep).

Tabel	Jumlah baris	Catatan
ingredient	24	25 kelas model dikurangi kelas "Image_9" (label nyasar dari dataset)
recipe	26	Masakan Indonesia sederhana
recipe_ingredient	136	91 bahan utama dan 45 bahan opsional
recipe_step	99	Langkah memasak
detection_history	1	Dari satu permintaan uji pada bagian 5.3

Logika rekomendasi: skor sebuah resep adalah jumlah bahan utama yang dimiliki dibagi total bahan utama resep tersebut. Resep diurutkan dari skor tertinggi, lalu yang paling sedikit kekurangan bahannya. Logika yang sama dipakai di server dan di aplikasi Flutter.

7. Hasil Pengujian Awal

Pengujian dilakukan berlapis: uji unit dan uji API otomatis, uji perbandingan (regresi) antar implementasi, dan uji manual terhadap server yang berjalan.

Pengujian	Cakupan	Hasil
pytest pada server	12 tes: kesehatan server, isi data awal, seed idempoten, daftar dan detail resep, rekomendasi, validasi masukan (400, 404, 422), riwayat	12 lolos
Perbandingan dengan implementasi acuan	80 kasus acak: urutan resep, skor, dan bahan kurang dari API dibandingkan dengan implementasi Python acuan	Identik
Server nyata (Gunicorn)	Seluruh endpoint dipanggil dengan curl, termasuk kasus galat	Berhasil (bagian 5.3)
Logika inti aplikasi Flutter (Dart)	758 pemeriksaan: pencocokan resep (80 kasus yang sama), letterbox, decode YOLO, penekanan duplikat (NMS)	Semua lolos
Model asli (best.onnx)	Tensor dari kode Dart dijalankan di model, hasil didecode kembali oleh Dart. Gambar uji sintetis	Mangga 69% dan Telur 58% tepat pada objek, sama dengan versi web

Daftar tes server (keluaran pytest -v):

```
test_health PASSED
test_seed_awal_sesuai_data PASSED
test_seed_idempoten PASSED
test_ingredients_tidak_memuat_kelas_abaian PASSED
test_daftar_dan_detail_resep PASSED
test_rekomendasi_dasar PASSED
test_rekomendasi_validasi PASSED
test_rekomendasi_nama_tidak_peka_huruf_besar PASSED
test_rekomendasi_sama_dengan_implementasi_acuan PASSED
test_riwayat PASSED
test_riwayat_validasi PASSED
test_alamat_tidak_ada PASSED
12 passed in 2.10s
```

7.1 Yang belum diuji

- Pembangunan dan pengujian aplikasi Flutter di HP sungguhan (kompilasi, kecepatan deteksi, izin kamera). Kode tampilan sudah ditulis tetapi belum dikompilasi.
- Koneksi dan perilaku PostgreSQL.
- Workflow CI di GitHub, karena repository belum di-push.
- Integrasi aplikasi dengan server (sinkronisasi resep dan pengiriman riwayat).
- Akurasi model pada foto kulkas nyata. Model saat ini mencapai mAP50 0,918 dan recall 0,906 pada data validasi, tetapi masih ada objek yang terlewat; training ulang dengan model lebih besar dan dataset gabungan sudah disiapkan.

8. Pembagian Tugas dan Rencana

Anggota	Peran	Tugas teknis
Pirni	Project Manager	Membuat repository dan aturan branch, papan tugas, CI, jadwal, koordinasi dan laporan
Nazwa Azzahra	Data Engineer	Penggabungan dataset di Roboflow, skema dan data awal database, pengembangan server
M. Denindra Pratama	ML Engineer	Training ulang YOLOv8, evaluasi recall per kelas, ekspor ONNX

Anggota	Peran	Tugas teknis
Shelva Meriana N	Developer Flutter	Build aplikasi di HP, perbaikan tampilan, integrasi dengan server
M. Rizky Akbar	QA dan Dokumentasi	Pengujian perangkat dan API, dokumentasi, materi presentasi

Langkah berikutnya

- Push repository ke GitHub dan isi tautannya pada sampul laporan ini.
- Bangun dan jalankan aplikasi Flutter di HP Android, catat kecepatan deteksi dan masalah yang muncul.
- Latih ulang model dengan dataset gabungan, lalu ganti best.onnx dan config.json di aplikasi.
- Pilih hosting server, uji PostgreSQL, lalu hubungkan aplikasi dengan server.

9. Risiko

Risiko	Dampak	Mitigasi
Aplikasi Flutter belum pernah dijalankan di HP	Error kompilasi atau performa lambat baru ketahuan belakangan	Jadwalkan uji di HP pada minggu ini. Logika inti sudah diuji terpisah sehingga perbaikan terbatas pada tampilan dan integrasi paket
Hosting gratis untuk server berubah kebijakan (sudah terjadi pada Hugging Face Docker)	Server tidak bisa dipublikasikan gratis	Deteksi dan resep sudah berjalan di perangkat sehingga aplikasi tetap berfungsi tanpa server. Bandingkan beberapa penyedia sejak awal
Ada bahan yang tidak terdeteksi	Rekomendasi resep kurang tepat	Pengguna dapat menambah atau menghapus bahan manual. Training ulang dan penambahan foto asli kulkas
Nama kelas model tidak cocok dengan data resep	Resep tidak muncul	Satu sumber nama kelas (config.json) dan pengecekan otomatis saat seed: bahan yang tidak dikenal menghentikan proses
Anggota berhalangan	Pekerjaan tertunda	Aturan pull request dan dokumentasi README di tiap folder memudahkan pengambilalihan